

04-web-basics

04. Web Basics (HTML/CSS/Tailwind)

Level: □ Beginner

Jam: 6 (3 minggu × 2 sesi)

Prasyarat: Modul 01 (JS Fundamentals)

Output: Landing page pribadi + Dashboard API live di Vercel

Tujuan Pembelajaran

Setelah modul ini, kamu bisa:

- Nulis HTML semantic dengan struktur header, nav, main, section, article, footer
- Layout pake CSS Flexbox & Grid
- Bikin responsive design tanpa framework
- Styling cepat pake Tailwind utility classes
- Implementasi dark mode pake Tailwind
- Manipulasi DOM pake JavaScript
- Fetch data dari API publik dan tampilin di halaman
- Deploy project ke Vercel

Materi

Sesi	Topik	File
1	HTML Semantic + CSS Flexbox/Grid + Responsive Design	01-html-css.md
2	Tailwind CSS + Dark Mode + Custom Config + Deploy	02-tailwind.md
3	DOM Manipulation + Events + Fetch API + Tampil Data	03-dom-fetch.md

Output Akhir Modul

Landing page pribadi + Dashboard API — Dua halaman web: landing page responsive pake HTML semantic +

Tailwind, dan dashboard yang nampilin data dari API publik.
Dua-duanya di-deploy ke Vercel.

AI Prompt Exercises

Sepanjang modul, latihan pake AI:

- "Explain this HTML structure: header > nav > ul > li * 3"
- "Buat CSS Grid 3 kolom dengan gap 16px"
- "Ubah layout ini dari Flexbox ke Grid"
- "Generate 3 Tailwind component: card, badge, alert"
- "Debug why my fetch is returning undefined"
- "Refactor this vanilla CSS to Tailwind classes"

4.1 HTML Semantic + CSS Flexbox/Grid + Responsive Design

HTML Semantic Tags

HTML semantic ngasih **makna** ke struktur halaman — bukan cuma `<div>` doang. Mesin pencari (SEO) dan screen reader pake ini buat ngerti konten.

```
<!DOCTYPE html>
<html lang="id">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Landing Page Saya</title>
</head>
<body>
  <header>
    <nav>
      <ul>
        <li><a href="#home">Home</a></li>
        <li><a href="#about">Tentang</a></li>
        <li><a href="#contact">Kontak</a></li>
      </ul>
    </nav>
  </header>

  <main>
    <section id="hero">
      <h1>Halo, Saya Budi</h1>
      <p>Web Developer & Designer</p>
```

```

</section>

<section id="about">
  <h2>Tentang Saya</h2>
  <p>Lulusan SMK jurusan RPL. Suka bikin web.</p>
</section>

<section id="projects">
  <h2>Project Saya</h2>
  <article>
    <h3>Project 1</h3>
    <p>Deskripsi project pertama.</p>
  </article>
  <article>
    <h3>Project 2</h3>
    <p>Deskripsi project kedua.</p>
  </article>
</section>
</main>

<footer>
  <p>&copy; 2026 Budi. All rights reserved.</p>
</footer>
</body>
</html>

```

Tag Semantic Penting

Tag	Fungsi
<header>	Bagian atas halaman (logo, navbar)
<nav>	Navigasi / menu
<main>	Konten utama (1x per halaman)
<section>	Kelompok konten yang related
<article>	Konten independen (blog post, card)
<aside>	Sidebar / konten samping
<footer>	Bagian bawah (copyright, link)

CSS Flexbox

Flexbox buat **satu dimensi** — baris ATAU kolom. Cocok buat navbar, card row, centering.

```

/* Parent: display: flex */
.container {
  display: flex;
  justify-content: center; /* horizontal: flex-start | center | space-between |
  align-items: center;      /* vertical: flex-start | center | stretch */
  gap: 16px;                /* jarak antar item */
  flex-wrap: wrap;          /* turun ke baris baru kalo penuh */
}

/* Child: flex item */
.item {
  flex: 1;                  /* bagi rata ruang */
  flex: 0 0 250px;          /* fixed width 250px */
  align-self: flex-end;     /* override alignment individual */
}

```

Contoh Navbar Flexbox

```

<style>
  nav {
    display: flex;
    justify-content: space-between;
    align-items: center;
    background: #333;
    color: white;
    padding: 16px 32px;
  }
  nav ul {
    display: flex;
    gap: 24px;
    list-style: none;
  }
  nav a { color: white; text-decoration: none; }
</style>

<nav>
  <div class="logo">MySite</div>
  <ul>
    <li><a href="#">Home</a></li>
    <li><a href="#">About</a></li>
    <li><a href="#">Contact</a></li>
  </ul>
</nav>

```

CSS Grid

Grid buat **dua dimensi** — baris DAN kolom sekaligus.

```
.grid-container {  
  display: grid;  
  grid-template-columns: 1fr 1fr 1fr; /* 3 kolom sama lebar */  
  grid-template-columns: repeat(3, 1fr); /* sama, lebih pendek */  
  grid-template-columns: 200px 1fr 1fr; /* kolom 1 fixed, 2 & 3 fleksibel */  
  gap: 16px;  
}
```

Contoh Card Grid

```
<style>  
  .card-grid {  
    display: grid;  
    grid-template-columns: repeat(auto-fit, minmax(250px, 1fr));  
    gap: 24px;  
    padding: 32px;  
  }  
  .card {  
    background: #f5f5f5;  
    padding: 24px;  
    border-radius: 8px;  
  }  
</style>
```

```
<section class="card-grid">  
  <div class="card">  
    <h3>HTML</h3>  
    <p>HyperText Markup Language – struktur web.</p>  
  </div>  
  <div class="card">  
    <h3>CSS</h3>  
    <p>Cascading Style Sheets – styling web.</p>  
  </div>  
  <div class="card">  
    <h3>JavaScript</h3>  
    <p>Interaktivitas dan logic di browser.</p>  
  </div>  
  <div class="card">  
    <h3>Tailwind</h3>  
    <p>Utility-first CSS framework.</p>  
  </div>  
</section>
```

Grid Areas (Layout Halaman Penuh)

```
.page-layout {  
  display: grid;  
  grid-template-areas:  
    "header header"  
    "sidebar main"  
    "footer footer";  
  grid-template-columns: 250px 1fr;  
  grid-template-rows: auto 1fr auto;  
  min-height: 100vh;  
}  
header { grid-area: header; }  
aside { grid-area: sidebar; }  
main { grid-area: main; }  
footer { grid-area: footer; }
```

Responsive Design & Media Queries

Breakpoint umum:

```
/* HP kecil: default (mobile-first) */  
body { font-size: 16px; }  
  
/* Tablet (≥ 768px) */  
@media (min-width: 768px) {  
  .container { max-width: 720px; margin: 0 auto; }  
}  
  
/* Desktop (≥ 1024px) */  
@media (min-width: 1024px) {  
  .container { max-width: 960px; }  
  .nav-menu { display: flex; }  
}
```

Contoh: Grid Mobile → Desktop

```
/* Mobile: 1 kolom */  
.project-grid {  
  display: grid;  
  grid-template-columns: 1fr;  
  gap: 16px;  
}
```

```

/* Tablet: 2 kolom */
@media (min-width: 768px) {
  .project-grid {
    grid-template-columns: repeat(2, 1fr);
  }
}

/* Desktop: 3 kolom */
@media (min-width: 1024px) {
  .project-grid {
    grid-template-columns: repeat(3, 1fr);
  }
}

```

Latihan

1. **Bikin layout halaman pribadi** — Struktur HTML semantic: header (nav), main (hero + about + projects), footer. Pakai CSS Flexbox buat navbar, Grid buat project cards.
2. **Bikin card grid responsive** — Grid dengan auto-fit dan min-max. Minimal 4 card. Masing-masing card ada gambar placeholder, judul, deskripsi.
3. **Bikin navbar mobile-friendly** — Navbar horizontal di desktop, berubah jadi vertikal/stack di HP (media query). Tambah hamburger menu (CSS-only, no JS).
4. **Bikin layout dashboard** — Sidebar kiri (nav), main area kanan (konten), header di atas. Pakai CSS Grid Areas. Responsive: sidebar sembunyi di mobile.

4.2 Tailwind CSS + Dark Mode + Custom Config + Deploy

Setup Tailwind via CDN (Cara Cepat)

Paling gampang buat latihan — tinggal tambahkan tag `<script>` di HTML.

```

<!DOCTYPE html>
<html lang="id">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Tailwind CDN</title>

```

```

    <script src="https://cdn.tailwindcss.com"></script>
  </head>
  <body class="bg-gray-100 p-8">
    <div class="max-w-2xl mx-auto bg-white rounded-xl shadow-md p-6">
      <h1 class="text-2xl font-bold text-gray-800">Halo Tailwind!</h1>
      <p class="mt-2 text-gray-600">Gak perlu nulis CSS manual.</p>
      <button class="mt-4 bg-blue-500 hover:bg-blue-600 text-white px-4 py-2 rounded">
        Klik Aku
      </button>
    </div>
  </body>
</html>

```

Utility Classes Wajib

Spacing (margin/padding)

Class	Value
p-4	padding: 16px
px-6	padding-x: 24px
py-2	padding-y: 8px
m-0	margin: 0
mx-auto	margin-x: auto
mt-8	margin-top: 32px
gap-4	gap: 16px

Flexbox & Grid

```

<!-- Flexbox -->
<div class="flex items-center justify-between gap-4">
  <div class="flex-1">Item 1</div>
  <div>Item 2</div>
</div>

<!-- Grid -->
<div class="grid grid-cols-1 md:grid-cols-3 gap-6">
  <div class="bg-white p-4 rounded shadow">Card 1</div>
  <div class="bg-white p-4 rounded shadow">Card 2</div>
  <div class="bg-white p-4 rounded shadow">Card 3</div>
</div>

```


Typography

```
<h1 class="text-3xl font-bold text-gray-900">Heading</h1>
<h2 class="text-2xl font-semibold text-gray-800">Subheading</h2>
<p class="text-base text-gray-600 leading-relaxed">
  Paragraf dengan line-height nyaman dibaca.
</p>
<span class="text-sm text-gray-400">Small text / caption</span>
```

Responsive Prefix

Prefix sm: md: lg: xl: dipake di depan utility class:

```
<!-- Mobile: 1 kolom, Desktop: 3 kolom -->
<div class="grid grid-cols-1 md:grid-cols-3 gap-4">

<!-- Mobile: text kecil, Desktop: text besar -->
<p class="text-sm md:text-lg lg:text-xl">Responsive text</p>

<!-- Mobile: hidden, Desktop: show -->
<div class="hidden md:block">Sidebar desktop</div>
```

Dark Mode

1. Aktifkan dark mode di config

Via CDN:

```
<script>
  tailwind.config = {
    darkMode: 'class', // pakai class strategy
  }
</script>
```

2. Toggle dark mode

```
<!DOCTYPE html>
<html lang="id">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Dark Mode</title>
  <script src="https://cdn.tailwindcss.com"></script>
  <script>
    tailwind.config = { darkMode: 'class' }
```

```

    </script>
  </head>
  <body class="bg-white dark:bg-gray-900 text-gray-900 dark:text-white min-h-screen">
    <div class="max-w-md mx-auto text-center">
      <h1 class="text-2xl font-bold mb-4">Dark Mode Demo</h1>
      <p class="mb-6 text-gray-600 dark:text-gray-400">Toggle untuk ganti tema.</p>

      <button id="toggleDark"
        class="bg-gray-800 dark:bg-white text-white dark:text-gray-800 px-6 py-2 rounded"
        > Toggle Dark Mode
      </button>

      <div class="mt-8 p-4 rounded-lg bg-gray-100 dark:bg-gray-800">
        <p class="text-gray-700 dark:text-gray-300">Konten ini ikut tema.</p>
      </div>
    </div>

    <script>
      const btn = document.getElementById('toggleDark');
      btn.addEventListener('click', () => {
        document.documentElement.classList.toggle('dark');
      });
    </script>
  </body>
</html>

```

Dark Mode Classes

```

<!-- Background -->
<div class="bg-white dark:bg-gray-800">

  <!-- Text -->
  <p class="text-gray-900 dark:text-gray-100">

    <!-- Border -->
    <div class="border border-gray-200 dark:border-gray-700">

      <!-- Card -->
      <div class="bg-white dark:bg-gray-800 shadow dark:shadow-gray-900">

```

Custom Config & Components

Custom colors & fonts

```
<script>
  tailwind.config = {
    darkMode: 'class',
    theme: {
      extend: {
        colors: {
          primary: '#3b82f6',
          secondary: '#8b5cf6',
          dark: {
            bg: '#0f172a',
            card: '#1e293b',
          }
        },
        fontFamily: {
          sans: ['Inter', 'sans-serif'],
        }
      }
    }
  }
</script>
```

Bikin Komponen Sendiri (dengan @apply)

Buat file style.css kalo mau pake @apply:

```
/* style.css */
@tailwind base;
@tailwind components;
@tailwind utilities;

@layer components {
  .btn-primary {
    @apply bg-blue-500 hover:bg-blue-600 text-white font-medium py-2 px-4 rounded
  }
  .card {
    @apply bg-white dark:bg-gray-800 rounded-xl shadow-md p-6;
  }
  .badge {
    @apply inline-block bg-blue-100 text-blue-800 text-xs font-medium px-2.5 py-
  }
}
```

Deploy ke Vercel

Manual (drag & drop)

1. Buka vercel.com
2. Login pake GitHub/GitLab/Email
3. Klik **Add New → Project**
4. Import repo atau seret folder project ke Vercel
5. **Build Command:** kosongin (kalo HTML statis)
6. **Output Directory:** . (root)
7. Klik **Deploy**

Via CLI

```
# Install Vercel CLI  
npm install -g vercel
```

```
# Deploy dari folder project  
vercel --prod
```

```
# Follow prompts – tinggal enter-enter
```

Project lo bakal live di `namaproject.vercel.app`.

Latihan

1. **Bikin landing page pribadi pake Tailwind** — Hero section (nama, tagline, CTA button), About section, Skills section (grid icons/badges). Responsive: 1 kolom mobile, 2 kolom tablet, 3 kolom desktop. Dark mode toggle.
2. **Bikin component library mini** — Pake `@apply` atau inline classes: Button (primary, secondary, outline), Card (dengan image placeholder, title, desc), Badge (info, success, warning, danger), Alert (success, error, info). Kumpulin semua dalam 1 halaman.
3. **Bikin pricing table** — 3 tier (Basic, Pro, Enterprise). Masing-masing ada harga, fitur list (dengan centang/cross), CTA button. Highlight tier "Popular" dengan warna beda. Responsive.
4. **Deploy ke Vercel** — Landing page dari latihan 1 di-deploy. Kirim URL hasilnya.

4.3 DOM Manipulation + Events + Fetch API + Tampil Data

DOM Manipulation

DOM (Document Object Model) — representasi HTML sebagai object JavaScript. Kita bisa ubah konten, atribut, style halaman secara dinamis.

Seleksi Element

```
// Single element
const judul = document.getElementById('judul');
const hero = document.querySelector('.hero'); // selector CSS
const btn = document.querySelector('#btn-submit');

// Multiple elements
const cards = document.querySelectorAll('.card'); // NodeList
const parag = document.getElementsByTagName('p'); // HTMLCollection
```

Ubah Konten

```
const judul = document.querySelector('h1');

judul.textContent = 'Halo Dunia!'; // ganti teks (aman, gak parse HTML)
judul.innerHTML = '<span>Halo</span>!'; // ganti teks + HTML (hati-hati XSS)
judul.innerText = 'Teks lain'; // kayak textContent tapi respect CSS
```

Ubah Style

```
const box = document.querySelector('.box');
box.style.backgroundColor = 'blue';
box.style.color = 'white';
box.style.padding = '20px';
box.style.borderRadius = '8px';

// Best practice: tambah/remove class, bukan inline style
box.classList.add('active');
box.classList.remove('hidden');
box.classList.toggle('dark-mode');
box.classList.contains('active'); // true/false
```

Manipulasi Atribut

```
const img = document.querySelector('img');
img.src = 'https://example.com/baru.jpg';
img.alt = 'Gambar baru';
img.setAttribute('loading', 'lazy');

const link = document.querySelector('a');
link.href = 'https://google.com';
link.target = '_blank';
```

Create & Append Element

```
// Bikin element baru
const card = document.createElement('div');
card.className = 'card';
card.textContent = 'Card baru';

// Tempelin ke parent
document.querySelector('.container').appendChild(card);

// Alternatif: insertAdjacentHTML (cepat buat banyak HTML)
document.querySelector('.list').insertAdjacentHTML('beforeend', `
  <li class="item">Item baru</li>
`);
```

Events

addEventListener

```
const btn = document.querySelector('#btn');

btn.addEventListener('click', () => {
  alert('Tombol diklik!');
});

// Event types umum:
// click, dblclick, mouseover, mouseout
// keydown, keyup, keypress
// submit, change, input
// load, DOMContentLoaded, scroll, resize
```

Contoh: Counter

```
<button id="minus">-</button>
<span id="count">0</span>
<button id="plus">+</button>

<script>
  let count = 0;
  const display = document.getElementById('count');

  document.getElementById('plus').addEventListener('click', () => {
    count++;
    display.textContent = count;
  });

  document.getElementById('minus').addEventListener('click', () => {
    count--;
    display.textContent = count;
  });
</script>
```

Event Delegation

Pake satu listener di parent — berguna buat element yang ditambah dinamis.

```
document.querySelector('.list').addEventListener('click', (e) => {
  const item = e.target.closest('.list-item');
  if (item) {
    console.log('Klik:', item.textContent);
    item.classList.toggle('selected');
  }
});
```

Form Handling

```
<form id="myForm">
  <input type="text" name="nama" placeholder="Nama" required>
  <input type="email" name="email" placeholder="Email" required>
  <button type="submit">Kirim</button>
</form>
<div id="output"></div>

<script>
  document.getElementById('myForm').addEventListener('submit', (e) => {
    e.preventDefault(); // biar gak reload halaman
```

```

const formData = new FormData(e.target);
const data = Object.fromEntries(formData); // { nama: "...", email: "..."}

document.getElementById('output').innerHTML = `
  <div class="alert alert-success">
    Halo ${data.nama}! Email kamu ${data.email}.
  </div>
`;
});
</script>

```

Fetch API & Tampil Data

Fetch Data dari API Publik

API publik gratis buat latihan:

- <https://jsonplaceholder.typicode.com/users> — data user
- <https://jsonplaceholder.typicode.com/posts> — data post
- <https://api.github.com/users/{username}> — data GitHub user
- <https://api.coindesk.com/v1/bpi/currentprice.json> — harga Bitcoin

```

fetch('https://jsonplaceholder.typicode.com/users')
  .then(response => {
    if (!response.ok) throw new Error('Network error');
    return response.json();
  })
  .then(data => {
    console.log(data); // array of users
  })
  .catch(error => {
    console.error('Fetch error:', error);
  });

```

Async/Await (lebih modern)

```

async function getUsers() {
  try {
    const res = await fetch('https://jsonplaceholder.typicode.com/users');
    if (!res.ok) throw new Error(`HTTP ${res.status}`);
    const users = await res.json();
    return users;
  }
}

```



```

    } catch (err) {
      console.error('Gagal fetch:', err);
      return [];
    }
  }
}

```

Tampilin ke Halaman

```

<div id="user-list" class="grid grid-cols-1 md:grid-cols-3 gap-4"></div>

<script>
  async function loadUsers() {
    const container = document.getElementById('user-list');
    container.innerHTML = '<p class="text-gray-500">Loading...</p>';

    try {
      const res = await fetch('https://jsonplaceholder.typicode.com/users');
      const users = await res.json();

      container.innerHTML = users.map(user => `
        <div class="bg-white p-4 rounded-lg shadow">
          <h3 class="font-bold text-lg">${user.name}</h3>
          <p class="text-gray-600 text-sm">@${user.username}</p>
          <p class="text-gray-500 mt-2">${user.email}</p>
          <p class="text-gray-400 text-sm">${user.company.name}</p>
        </div>
      `).join('');
    } catch (err) {
      container.innerHTML = `<p class="text-red-500">Error: ${err.message}</p>`;
    }
  }

  loadUsers();
</script>

```

Contoh: Dashboard Lengkap

```

<!DOCTYPE html>
<html lang="id">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Dashboard API</title>
  <script src="https://cdn.tailwindcss.com"></script>
</head>

```

```

<body class="bg-gray-100 min-h-screen p-8">
  <div class="max-w-6xl mx-auto">
    <h1 class="text-3xl font-bold mb-6">Dashboard</h1>

    <!-- Controls -->
    <div class="flex gap-4 mb-6">
      <button id="btnUsers" class="bg-blue-500 text-white px-4 py-2 rounded">Tam
      <button id="btnPosts" class="bg-green-500 text-white px-4 py-2 rounded">Ta
    </div>

    <!-- Loading indicator -->
    <div id="loading" class="hidden text-gray-500 mb-4">Memuat data...</div>

    <!-- Output -->
    <div id="output" class="grid grid-cols-1 md:grid-cols-2 lg:grid-cols-3 gap-4">
  </div>

  <script>
    const output = document.getElementById('output');
    const loading = document.getElementById('loading');

    async function fetchData(endpoint) {
      loading.classList.remove('hidden');
      output.innerHTML = '';

      try {
        const res = await fetch(`https://jsonplaceholder.typicode.com/${endpoint}`);
        const data = await res.json();

        loading.classList.add('hidden');

        if (endpoint === 'users') {
          output.innerHTML = data.map(u => `
            <div class="bg-white rounded-xl shadow p-4">
              <h3 class="font-bold text-lg">${u.name}</h3>
              <p class="text-gray-600">@${u.username}</p>
              <p class="text-sm text-gray-500">${u.email}</p>
              <p class="text-xs text-gray-400 mt-2">${u.address.city}</p>
            </div>
          `).join('');
        } else {
          output.innerHTML = data.slice(0, 12).map(p => `
            <div class="bg-white rounded-xl shadow p-4">
              <h3 class="font-bold">${p.title.slice(0, 40)}...</h3>
              <p class="text-sm text-gray-600 mt-2">${p.body.slice(0, 80)}...</p>
            </div>
          `).join('');
        }
      } catch (error) {
        console.error('Error fetching data:', error);
      }
    }

    // Initial fetch
    fetchData('users');
  </script>

```

```

        `).join('');
    }
  } catch (err) {
    loading.classList.add('hidden');
    output.innerHTML = `<p class="text-red-500 col-span-3">Error: ${err.message}</p>`;
  }
}

document.getElementById('btnUsers').addEventListener('click', () => fetchData);
document.getElementById('btnPosts').addEventListener('click', () => fetchData);
</script>
</body>
</html>

```

Loading & Error State

Selalu handle 3 state:

1. **Loading** — tampilkan spinner/"Memuat..."
2. **Success** — tampilkan data
3. **Error** — tampilkan pesan error

```

function renderStates() {
  // Loading
  container.innerHTML = `<div class="spinner">Loading...</div>`;

  // Success
  container.innerHTML = data.map(item => `<div>${item.name}</div>`).join('');

  // Error
  container.innerHTML = `<div class="error">Gagal memuat: ${err.message}</div>`;
}

```

Latihan

1. **Todo List App** — Input buat nambah todo, tombol delete, toggle selesai/belum. Pake createElement,.classList.toggle, addEventListener. Data disimpan di array local (belum localStorage).
2. **GitHub User Search** — Input username, fetch <https://api.github.com/users/{username}> tampilkan avatar, name, bio, public repos, followers/following. Handle error kalo user gak ditemukan (404).
3. **Dashboard API dengan filter** — Fetch dari /users dan /posts. Bikin filter: users by city, posts by userId. Tampilkan loading spinner, handle error, tampilkan jumlah data yang difilter.

4. **Deploy dashboard ke Vercel** — Ambil salah satu latihan di atas, deploy ke Vercel. Pastikan loading & error state berfungsi. Kirim URL.